

Name: Kirsten Kajubi
Course: CIS 310
Assignment 3: 8 bit processor
Date: 04/20/2026

Github Repo Link: <https://github.com/KirstenKajubi/Assignment-3-8-bit-processor>

Title: Design and Implementation of an 8-Bit Processor in Digital

1. Introduction

In this project, I designed and implemented an 8-bit processor using the Digital logic simulator. The goal was to build a working CPU that could fetch, decode, execute, and write back instructions correctly. The design combines a program counter, instruction memory, instruction register, control unit, register file, ALU, and supporting datapath components.

The processor was built as a multicycle design. Each instruction moves through four stages: fetch, decode, execute, and writeback. This allowed me to separate timing and datapath behavior clearly and made it easier to debug the system one stage at a time.

The final processor supports both R-type and I-type instructions and uses a structured instruction format so that different operations can be selected using opcode bits and operand fields.

2. Instruction Format

The processor uses an 8-bit instruction format. The instruction is divided into four fields:

- Bit 7: T bit
- Bits 6 to 4: opcode
- Bits 3 to 2: destination register Rd
- Bits 1 to 0: source register Rs or immediate value

This means the processor supports both register-based and immediate-based operations.

For R-type instructions:

- T = 0
- bits 1 to 0 select the source register

For I-type instructions:

- T = 1
- bits 1 to 0 represent a 2-bit immediate value

3. Supported Instructions

The processor supports the following operations.

R-type instructions:

- ADD Rd, Rs
- ADDC Rd, Rs
- SUBB Rd, Rs
- SUB Rd, Rs
- PASS Rd
- INC Rd
- DEC Rd

I-type instruction:

- LDI Rd, #imm

These operations are selected using the 3-bit opcode field.

4. Processor Architecture

The final processor consists of the following main components.

4.1 Program Counter (PC)

The PC stores the address of the current instruction. It increments only during the fetch stage. A 2-bit control input is used to decide whether the PC holds or increments.

4.2 Instruction Memory

Instruction memory stores the program being executed. It outputs an 8-bit instruction based on the current PC address.

4.3 Instruction Register (IR)

The IR stores the currently fetched instruction. It loads only during the fetch stage and holds the instruction stable while the processor moves through decode, execute, and writeback.

4.4 Control Unit

The control unit is implemented as a 4-stage one-hot state machine. Its stages are:

- inst_fetch

- inst_decode
- inst_exec
- write_back

From these stages, the final control signals are generated:

- IR_load = inst_fetch
- PC_inc = inst_fetch
- RegWrite = write_back

This allows the processor to fetch a new instruction only when appropriate and write results back only during the final stage of the instruction cycle.

4.5 Register File

The processor uses a 4-register, 8-bit register file containing:

- R0
- R1
- R2
- R3

The register file has:

- two read ports
- one write port

Inputs:

- ReadSelA
- ReadSelB
- WriteSel
- WriteData
- WriteEnable
- clk

Outputs:

- ReadDataA
- ReadDataB

4.6 ALU

The final ALU is an 8-bit ALU controlled by a 3-bit operation input. It supports all required assignment operations:

- 000 = ADD
- 001 = ADDC
- 010 = SUBB
- 011 = SUB
- 100 = PASS / load-immediate behavior
- 101 = INC
- 110 = DEC

The ALU takes:

- A input from the destination register
- B input from either the source register or immediate path

4.7 Immediate Path

The lower 2 bits of the instruction are zero-extended to 8 bits for I-type instructions. This is done by combining a 6-bit zero constant with the 2-bit immediate field.

4.8 ALU Input Mux

A 2-to-1 mux selects the ALU B input:

- register source for R-type instructions
- immediate value for I-type instructions

The select line for this mux is the T bit from the instruction.

5. Datapath Operation

The final datapath can be described as:

PC -> Instruction Memory -> IR

IR -> splitters -> T, opcode, Rd, Rs/imm

Rd -> RegisterFile ReadSelA

Rs/imm -> RegisterFile ReadSelB

Rd -> RegisterFile WriteSel

RegisterFile ReadDataA -> ALU A
RegisterFile ReadDataB -> ALU B mux input 0
Immediate zero-extended value -> ALU B mux input 1

T -> ALU B mux select
opcode -> ALU control
ALU result -> RegisterFile WriteData
RegWrite -> RegisterFile WriteEnable

This structure allows the processor to support both register operations and immediate operations using the same ALU.

6. Final Control Behavior

The control unit uses a 4-stage instruction cycle:

- Fetch: instruction is loaded into the IR and PC increments
- Decode: instruction fields are interpreted
- Execute: ALU performs the requested operation
- Writeback: result is stored into the selected register

Only one stage is active at a time. This one-hot sequencing helped ensure correct synchronization between memory, control, and datapath components.

7. Test Program

The provided test program contains eight instructions:

- LDI R1, #3
- LDI R2, #1
- ADD R1, R2
- SUB R1, R2
- INC R1
- DEC R1
- ADDC R1, R2
- PASS R1

The corresponding decimal machine-code values are:

- 143
- 147
- 12
- 108

-
- The screenshot displays a digital logic circuit simulation for an 8-bit processor. The interface includes a menu bar (File, Edit, View, Simulation, Analysis, Components, Windows, Help) and a toolbar with various simulation and logic tools. The circuit components and their connections are as follows:
- PC (Program Counter):** An 8-bit counter that receives a clock signal (clk) and a load signal. Its output (PC_output) is connected to the instruction register (IR).
 - IR (Instruction Register):** An 8-bit register that stores the instruction fetched from memory. Its output (IR_output) is connected to the ALU and the instruction decoder.
 - ALU (Arithmetic Logic Unit):** An 8-bit ALU that performs operations on data from the registers and the instruction register. Its output (ALU_output) is connected to the registers.
 - Registers:** Two 8-bit registers, RegA and RegB, which store data. They are connected to the ALU output and the instruction register.
 - Control Logic:** A central control unit that manages the flow of data and instructions. It includes components like the instruction decoder, ALU control, and register control.
 - Memory:** A memory block that stores instructions. It is connected to the PC output and the instruction register.
- The circuit is simulated using a logic simulator, showing the internal logic and data flow of the processor. The simulation results are displayed in the bottom window, showing the values of the various signals and components over time.

8. Design Challenges and Debugging

Several major issues had to be resolved during the project.

First, the original early design was too simple and only supported a single accumulator-style register. This had to be redesigned into a real register file to support the assignment ISA.

Second, instruction memory behavior had to be stabilized so that the IR would not reload incorrectly during every cycle. This was solved by connecting IR_load only to the fetch stage.

Third, the PC had to be redesigned so it would only increment during fetch instead of every clock cycle. This ensured that each instruction stayed stable in the IR through all four stages.

Fourth, the ALU had to be redesigned to support multiple operations instead of only simple addition. A final optimized ALU was built using shared arithmetic paths and a mux-based result selection structure.

Fifth, instruction splitting had to be corrected carefully so that T, opcode, Rd, and Rs/imm were all extracted from the correct bit positions.

9. Conclusion

This project successfully demonstrates the design and implementation of an 8-bit multicycle processor in Digital. The final processor includes a structured datapath, a proper register file, a multi-operation ALU, and a timing-based control unit. It supports both R-type and I-type instructions and is able to execute the provided test program.

The project helped reinforce key computer architecture concepts such as instruction decoding, control signal generation, register file design, ALU design, immediate handling, and multicycle timing